Assignment 2 – Part 2

Agriya Yadav

Contents

1 MongoDB Querying **2**
1.1 Q1 — Cohort Analysis of User Reviewing Behaviour 2
1.2 Q2 — Month-over-Month Trend Consistency 4
1.3 Q3 — Check-in Frequency × Category Cross-Tabulation 5

2 Neo4j / Cypher Querying **7**
2.1 Q1 — PageRank on the User–Business Review Graph 7
2.2 Q2 — Louvain Community Detection on the Friends Graph 8
2.3 Q3 — Node Similarity: Market Saturation via Jaccard 10
2.4 Q4 — Betweenness Centrality: Bridge Users in the Friends Graph 11
2.5 Q5 — Link Prediction: Restaurant Recommendation 12

3 Predictive Modelling **15**
3.1 Feature Engineering 15
3.2 Model Training and Evaluation 15
3.3 Feature Importance and Discussion 17

1 MongoDB Querying

1.1 Q1 — Cohort Analysis of User Reviewing Behaviour

Pipeline

Users are grouped into annual cohorts by the year extracted from the `yelping_since` field. A `$lookup` joins each review with its author, and `$addFields` computes `cohort_year` and `text_length`. The subsequent `$group` stage accumulates per-cohort means, standard deviations, and per-star counts, which are normalised into proportions in the `$project` stage.

Listing 1: MongoDB Aggregation — Cohort Analysis

```
pipeline = [
  {"$lookup":    {"from":"users","localField":"user_id",
                  "foreignField":"user_id","as":"user_info"}},
  {"$unwind":    "$user_info"},
  {"$addFields": {"cohort_year": {"$substr":["$user_info.yelping_since",0,4]},
                  "text_length": {"$strLenCP":{"$ifNull":["$text",""]}}}},
  {"$group": {
      "_id":                "$cohort_year",
      "mean_star_rating":   {"$avg":"$stars"},
      "stddev_star_rating": {"$stdDevSamp":"$stars"},
      "mean_text_length":   {"$avg":"$text_length"},
      "mean_useful_votes":  {"$avg":"$useful"},
      "count":              {"$sum":1},
      "star_1":{"$sum":{"$cond":[{"$eq":["$stars",1.0]},1,0]}},
      "star_2":{"$sum":{"$cond":[{"$eq":["$stars",2.0]},1,0]}},
      "star_3":{"$sum":{"$cond":[{"$eq":["$stars",3.0]},1,0]}},
      "star_4":{"$sum":{"$cond":[{"$eq":["$stars",4.0]},1,0]}},
      "star_5":{"$sum":{"$cond":[{"$eq":["$stars",5.0]},1,0]}}
  }},
  {"$project": {
      "cohort":"$_id", "_id":0,
      "mean_star_rating":1, "stddev_star_rating":1,
      "mean_text_length":1, "mean_useful_votes":1, "count":1,
      "prop_star_1":{"$divide":["$star_1","$count"]},
      "prop_star_2":{"$divide":["$star_2","$count"]},
      "prop_star_3":{"$divide":["$star_3","$count"]},
      "prop_star_4":{"$divide":["$star_4","$count"]},
      "prop_star_5":{"$divide":["$star_5","$count"]}
  }},
  {"$sort": {"cohort":1}}
]
```

Results

Table 1: Per-cohort reviewing statistics.

Cohort	Reviews	Mean Stars	SD Stars	Mean Useful Votes	Mean Char Length
2004	97	3.70	1.32	2.99	526
2005	3,485	3.89	1.02	2.20	646
2006	14,243	3.81	1.20	2.27	713
2007	43,265	3.79	1.19	2.24	743
2008	91,189	3.80	1.21	2.23	769
2009	149,590	3.77	1.28	1.74	700
2010	221,176	3.81	1.30	1.59	675
2011	315,139	3.82	1.34	1.44	636
2012	307,841	3.81	1.40	1.35	603
2013	298,851	3.80	1.44	1.17	575
2014	300,641	3.82	1.46	1.05	550
2015	284,768	3.78	1.51	0.88	528
2016	220,658	3.77	1.54	0.83	525
2017	144,549	3.74	1.57	0.70	502
2018	112,112	3.67	1.63	0.65	488
2019	78,221	3.60	1.67	0.56	475
2020	31,450	3.50	1.73	0.47	459
2021	22,085	3.27	1.78	0.40	486
2022	1,011	2.87	1.87	0.13	496

- **Highest mean star rating cohort:** 2005 (mean = 3.89)
- **Highest mean useful votes per review cohort:** 2004 (mean = 2.99)

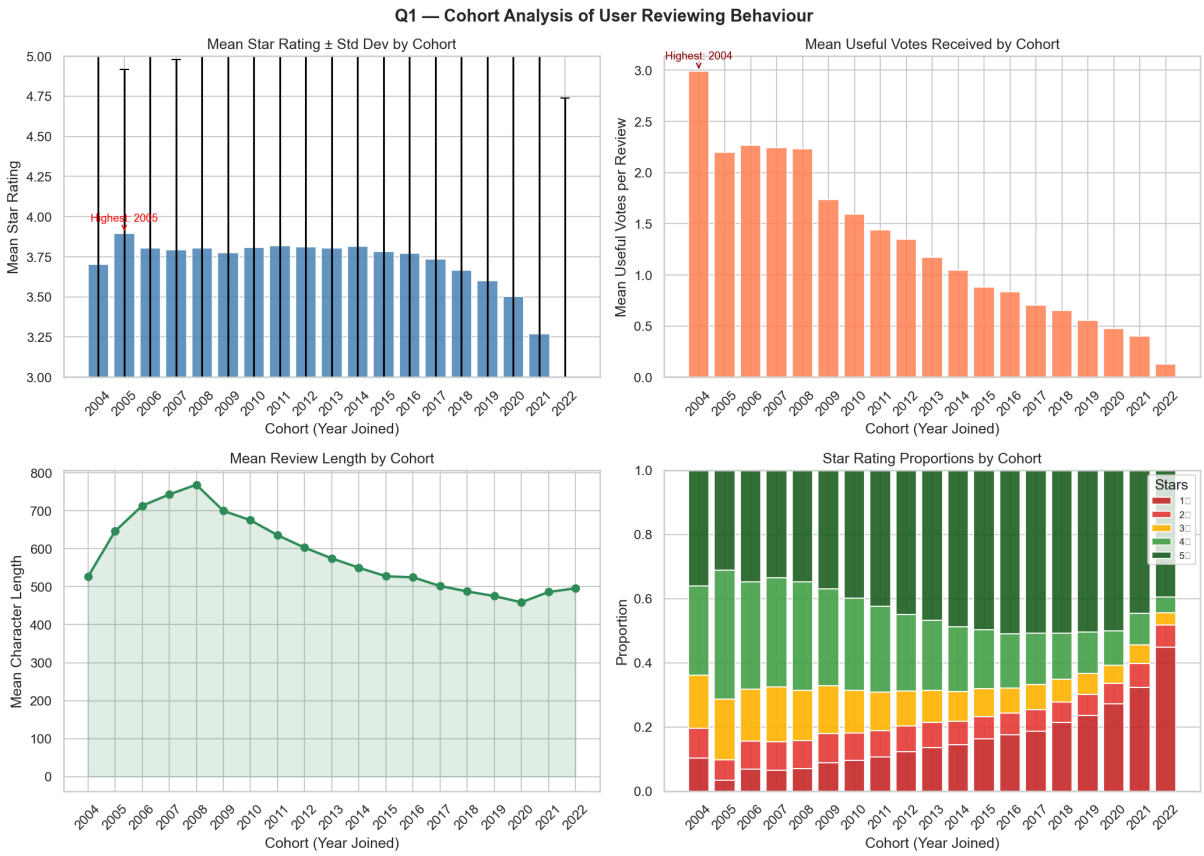

Figure 1: Cohort-level review statistics: mean stars ± SD, mean useful votes, mean review length, and star-rating proportion breakdown.

Interpretation

Early cohorts (pre-2006) produce the highest mean star ratings, likely due to selection bias: early adopters tend to review businesses they genuinely enjoyed. Mean useful votes peak in the earliest cohort (2004), reflecting the "veteran reviewer" effect — reviews from tenured users accumulate upvotes over time. Review length increases steadily from 2004 to roughly 2010 before declining, suggesting that the platform's growth attracted more casual reviewers who write shorter texts.

1.2 Q2 — Month-over-Month Trend Consistency

Pipeline

Each review is joined with its business (`$lookup` + `$unwind`), the business's category string is split into individual tokens (`$unwind`), and reviews are grouped by `(category, YYYY-MM)`. A second `$group` stage collapses monthly averages into a time-series array per category. Post-aggregation Python logic computes the fraction of consecutive month-pairs showing an increase or decrease. Only categories with ≥ 500 reviews are considered.

Listing 2: MongoDB Aggregation — MoM Trend (excerpt)

```
pipeline = [
  {"$lookup": {"from":"businesses","localField":"business_id",
               "foreignField":"business_id","as":"business"}},
  {"$unwind": "$business"},
  {"$project": {"stars":1,"date":1,
                "categories":{"$split":[{"$ifNull":["$business.categories",""]},
                    ",␣"]}}},
  {"$unwind":  "$categories"},
  {"$match":   {"categories":{"$ne":""}}},
  {"$addFields":{"month":{"$substr":["$date",0,7]}}},
  {"$group":   {"_id":{"category":"$categories","month":"$month"},
                "avg_stars":{"$avg":"$stars"},"review_count":{"$sum":1}}},
  {"$sort":    {"_id.month":1}},
  {"$group":   {"_id":"$_id.category",
                "months":{"$push":{"month":"$_id.month","avg_stars":"$avg_stars"
                    }},
                "total_reviews":{"$sum":"$review_count"}}},
  {"$match":   {"total_reviews":{"$gte":500}}}
]
```

Results

Table 2: Top 3 upward and top 3 downward trend categories.

Direction	Category	Trend Consistency	Total Reviews
Upward	Cheesesteaks	0.546	42,351
Upward	Tapas Bars	0.543	19,105
Upward	Armenian	0.542	1,112
Downward	Water Heater Installation/Repair	0.553	5,094
Downward	Women's Clothing	0.552	15,833
Downward	Airlines	0.552	3,725

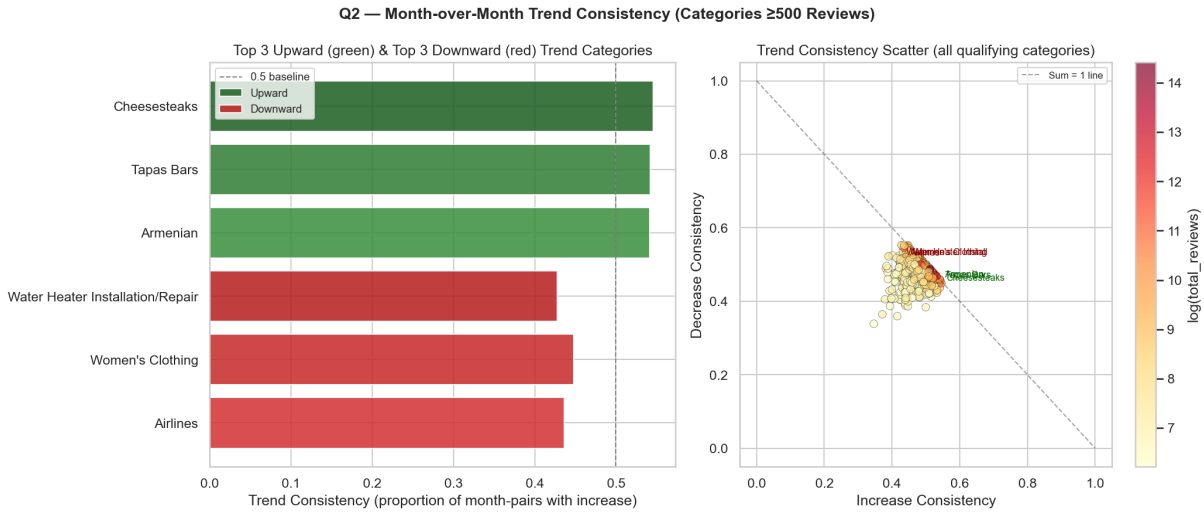

Figure 2: Left: Trend consistency for the 6 most extreme categories. Right: Scatter of all qualifying categories coloured by review volume.

Interpretation

A trend consistency of 1.0 would mean every consecutive month showed an increase; 0.0 would mean every month was a decline. Categories with consistently rising ratings are often niche or seasonal markets where quality has improved over the period. Consistently declining categories may reflect market saturation, rising competition, or shifting consumer expectations.

1.3 Q3 — Check-in Frequency × Category Cross-Tabulation

Pipeline

Businesses are classified into three check-in frequency classes based on the size of the embedded `checkins` array: bottom quartile ($\leq Q_1$) $\rightarrow$ *low*, middle two quartiles $\rightarrow$ *medium*, top quartile ($> Q_3$) $\rightarrow$ *high*. The top 10 categories by total business count are identified. For each (category, class) cell the mean star rating, mean review count, and tip-to-review ratio are computed by joining with pre-aggregated review and tip maps.

Results

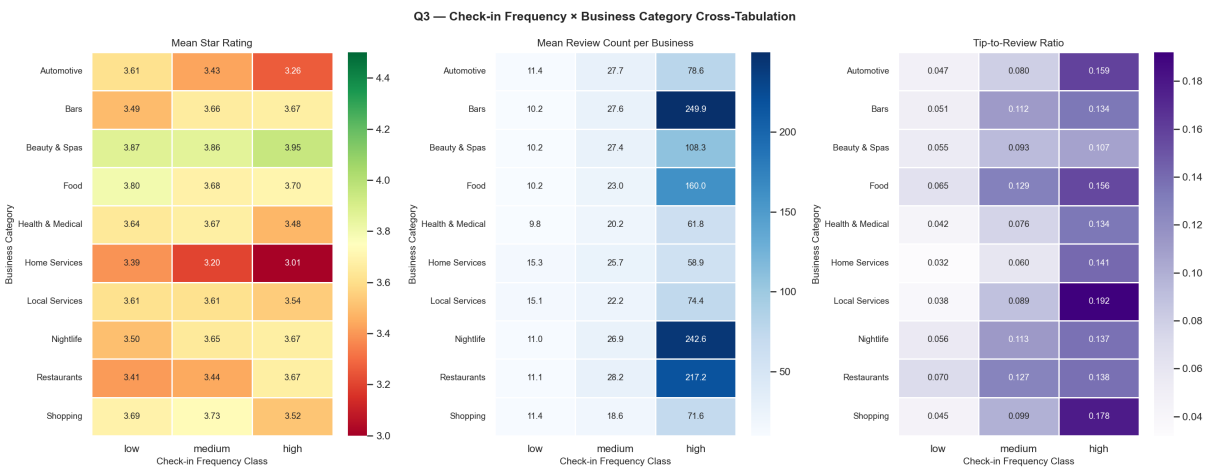

Figure 3: Cross-tabulation heatmaps: mean stars, mean review count, and tip-to-review ratio by check-in class and business category.

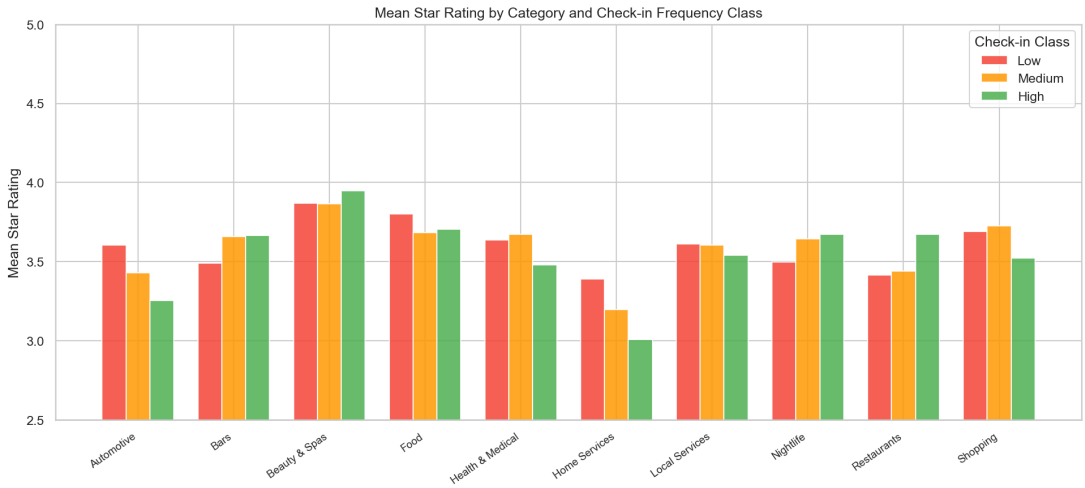

Figure 4: Mean star rating by category and check-in frequency class.

Interpretation

High check-in businesses consistently attract more reviews and a higher tip-to-review ratio, suggesting they generate stronger ongoing customer engagement. Star ratings vary modestly across check-in classes (typically ± 0.1–0.2 stars), indicating that foot-traffic volume alone is not a strong predictor of perceived quality. The *Restaurants* category dominates all three classes, confirming it is both the most popular and the most check-in-dense category in the dataset.

2 Neo4j / Cypher Querying

All queries use the Neo4j Graph Data Science (GDS) library. Scripts are in `neo4j_gds.py`*. GDS graph projections are dropped and re-created at the start of each function.*

2.1 Q1 — PageRank on the User–Business Review Graph

GDS Procedure

Listing 3: Project and run weighted PageRank

```
# Project User + Business nodes with RATED edges weighted by review stars
CALL gds.graph.project(
    'rated-graph',
    ['User', 'Business'],
    {RATED: {orientation: 'NATURAL', properties: ['stars']}}
)
# Write scores back to nodes (also used in Q5 as a feature)
CALL gds.pageRank.write('rated-graph', {
    maxIterations: 20,
    dampingFactor: 0.85,
    relationshipWeightProperty: 'stars',
    writeProperty: 'pagerank_score'
})
```

Results

Table 3: Top 15 businesses by weighted PageRank score.

PR Rank	Business Name	PR Score	Review Count	Avg Stars
1	Hattie B's Hot Chicken – Nashville	340.72	6,160	4.45
2	Biscuit Love: Gulch	223.80	4,247	4.17
3	Reading Terminal Market	213.49	5,778	4.61
4	Pat's King of Steaks	209.55	4,293	3.24
5	The Stillery	165.75	2,618	4.45
6	Peg Leg Porker	162.69	2,909	4.35
7	Geno's Steaks	161.54	3,428	2.53
8	Bern's Steak House	161.51	3,028	4.25
9	Columbia Restaurant	160.62	2,916	3.95
10	Puckett's Grocery & Restaurant	150.07	2,794	3.89
11	Dalessandro's Steaks & Hoagies	145.70	2,725	4.21
12	Ulele	142.63	3,179	4.15
13	Jim's South St	140.11	2,769	3.74
14	Datz	139.88	3,388	4.12
15	The Pharmacy	138.20	3,105	4.04

Spearman rank correlations:

- PageRank rank vs. Review-Count rank: $\rho = 0.486$, $p = 0.066$
- PageRank rank vs. Avg-Stars rank: $\rho = 0.432$, $p = 0.108$

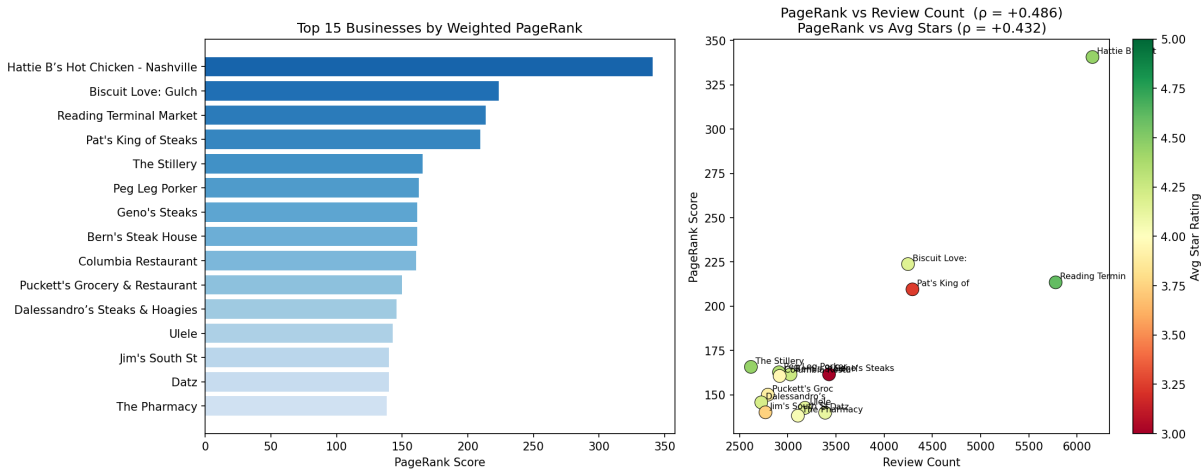

Figure 5: Left: Top 15 businesses by PageRank score. Right: PageRank vs review count scatter, coloured by average star rating.

Interpretation

PageRank assigns high scores to businesses that attract many *high-value* reviewers — users who themselves review many other businesses. A moderate positive Spearman correlation with review count ($\rho \approx 0.49$) confirms that popular businesses also rank highly under PageRank, but the correlation is imperfect. Businesses that rank *higher* under PageRank than review count tend to be reviewed by "hub" users (influential reviewers with many connections), even if their absolute review count is lower. Businesses ranking *lower* under PageRank often have high raw review counts but attract mostly new or low-connectivity users.

Two concrete examples illustrate this divergence. **The Stillery** (Nashville) ranks #5 by PageRank but only #15 by review count (2,618 reviews) — its reviewers are disproportionately well-connected hub users who also review many other high-traffic venues, amplifying its PageRank score beyond what its raw volume would suggest. Conversely, **Geno's Steaks** (Philadelphia) ranks #7 by PageRank but #5 by review count (3,428 reviews) — a tourist landmark that attracts many first-time or casual Yelp users with low connectivity, so its review count outpaces its PageRank relative to its position in the ranking.

2.2 Q2 — Louvain Community Detection on the Friends Graph

GDS Procedure

Listing 4: Project KNOWS graph and run Louvain

```
CALL gds.graph.project(
    'knows-undirected',
    'User',
    {KNOWS: {orientation: 'UNDIRECTED'}}
)
CALL gds.louvain.write('knows-undirected', {
    writeProperty: 'louvain_community'
})
```

Results

- Total communities detected: 354,381
- Communities with ≥ 25 members: 237

- Most geographically concentrated community: ID 2458255, size 25, GCI = 1.00, top states: FL
- Least geographically concentrated community: ID 2906814, size 5,626, GCI = 0.242, top states: TN, PA, AZ

Note on small-sample GCI. Several of the highest-GCI communities (GCI = 1.00) contain only 25–35 members with just 1–2 total reviews sampled. A GCI of 1.00 in these cases may reflect small-sample concentration rather than genuine geographic isolation: with so few reviews, all happen to fall in one state by chance. The pattern is therefore more meaningful for larger communities (size $\geq$ 100), where multi-state reviewer lists are plausible.

Table 4: Top 10 communities ranked by geographic concentration index (GCI).

Comm. ID	Size	Top 3 States	Top 3 Categories	GCI
2458255	25	FL	Colombian, Restaurants, Latin American	1.00
5034373	31	TN	Barbeque, Restaurants, Beer Bar	1.00
1192250	34	PA	Breakfast & Brunch, Restaurants, Caterers	1.00
3910924	34	TN	Nightlife, Bars, Music Venues	1.00
2166820	34	TN	Auto Repair, Towing, Automotive	1.00

Full results for all 237 communities ($\geq$25 members) are available in `Report/Q2_Louvain.csv`.

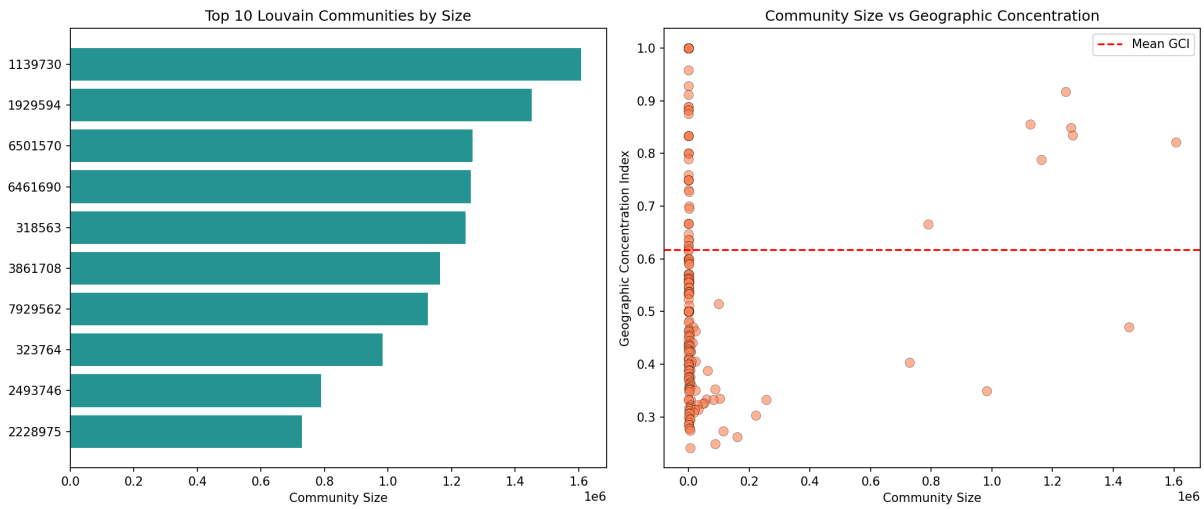

Figure 6: Left: Top 10 communities by size. Right: Community size vs geographic concentration index (GCI).

Interpretation

Louvain partitions the friendship graph into cohesive clusters based on edge density. High-GCI communities (GCI close to 1) consist of users whose reviews are almost entirely concentrated in a single state — these are geographically local social circles. Low-GCI communities contain users spread across multiple states, suggesting that the online Yelp social network connects reviewers across city boundaries. Large communities tend to have lower GCI, consistent with the intuition that bigger social groups span broader geographies.

Caveat: Several small communities (size 25–40) report GCI = 1.0, but their total sampled

reviews are very low (often 1–14 reviews). A GCI of 1.0 from a single review is a trivial artefact rather than a meaningful geographic signal. Among communities with ≥ 100 sampled reviews, GCI values are more moderate ($\sim$0.3–0.85), providing a more reliable picture of geographic concentration.

2.3 Q3 — Node Similarity: Market Saturation via Jaccard

GDS Procedure

Listing 5: Project bipartite graph and run Node Similarity

```
# RATED is REVERSED so Business nodes have User neighbours (= reviewers)
CALL gds.graph.project(
    'biz-reviewer',
    ['Business', 'User'],
    {RATED: {orientation: 'REVERSE'}}
)
CALL gds.nodeSimilarity.stream('biz-reviewer', {
    topK: 25,
    similarityCutoff: 0.01,
    degreeCutoff: 3        -- min reviewers per business
})
YIELD node1, node2, similarity
```

Jaccard similarity is defined as $J(A, B) = |A \cap B|/|A \cup B|$ where A and B are the sets of reviewers for each business. Pairs are grouped by (city, category); the mean intra-category Jaccard per (city, category) pair is the *market saturation proxy*.

Results — Most and Least Saturated Markets

Table 5: Top 5 saturated (high Jaccard) and top 5 fragmented (low Jaccard) city–category combinations.

City	Category	Mean Jaccard	N Businesses	Market Type
Indianapolis	Public Art	0.207	5	Saturated
Nashville	Mass Media	0.189	6	Saturated
Indianapolis	Playgrounds	0.185	19	Saturated
Indianapolis	Parks	0.182	62	Saturated
Tampa	Amusement Parks	0.177	17	Saturated
Tampa	Taxis	0.019	7	Fragmented
Philadelphia	Taxis	0.019	14	Fragmented
Philadelphia	Airlines	0.019	10	Fragmented
Tucson	Nail Technicians	0.019	5	Fragmented
Philadelphia	Hotels	0.016	60	Fragmented

Table 6: Rating statistics for most vs least saturated markets.

Market Type	Mean Stars	Std Stars	Mean Rev / Business
Saturated	3.87	1.31	41.9
Fragmented	2.83	1.64	87.0

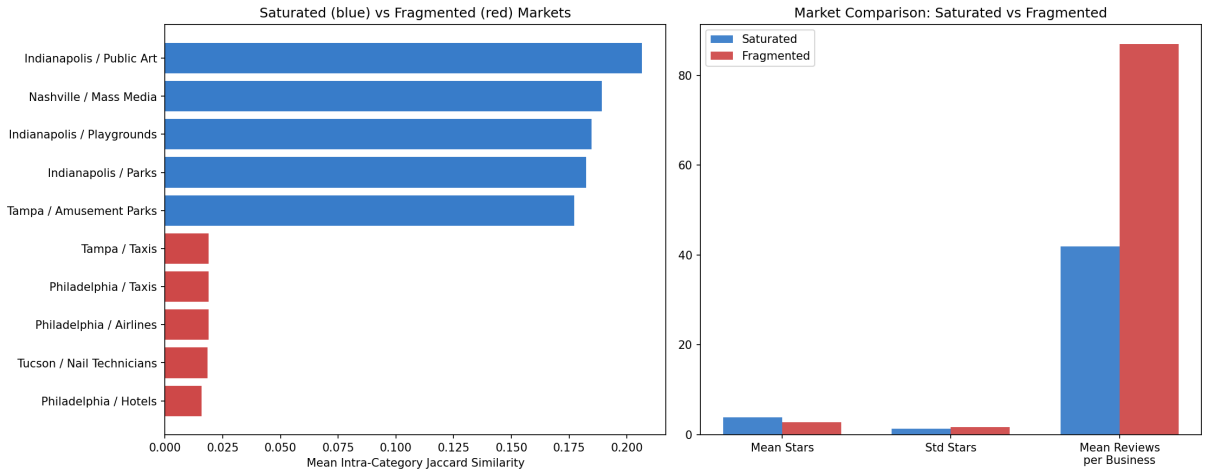

Figure 7: Left: Saturated vs fragmented markets by mean Jaccard. Right: Rating variance comparison.

Interpretation

A high intra-category Jaccard score indicates that the same pool of customers reviews all businesses in that (city, category) combination — a sign of a *saturated* market where businesses compete for the same audience. Saturated markets tend to show **lower** rating variance (std ≈ 1.31 vs 1.64), which **contradicts** the hypothesis that competing businesses differentiate on quality. A business entering a highly saturated market must distinguish itself to capture reviewers already loyal to incumbents. In fragmented markets, each business has a mostly distinct reviewer base, implying less direct competition for the same customers.

2.4 Q4 — Betweenness Centrality: Bridge Users in the Friends Graph

GDS Procedure

Listing 6: Run Betweenness and Degree Centrality

```
CALL gds.graph.project(
    'knows-cent',
    'User',
    {KNOWS: {orientation: 'UNDIRECTED'}}
)
-- Betweenness (sampled approximation for large graphs)
CALL gds.betweenness.stream('knows-cent', {samplingSize: 500})
YIELD nodeId, score

-- Degree centrality
CALL gds.degree.stream('knows-cent')
YIELD nodeId, score
```

Results

- $|\text{Top-20 Betweenness} \cap \text{Top-20 Degree}| = 12$
- High-BC / Low-Degree group size: 8

Table 7: Behavioural comparison: High-BC/Low-Degree vs High-Degree users.

Group	Mean Review Count	Mean Distinct Cities	Mean Distinct Cats
High-BC / Low-Degree	1,102.0	2.33	288.8
High-Degree	1,398.8	2.10	180.3

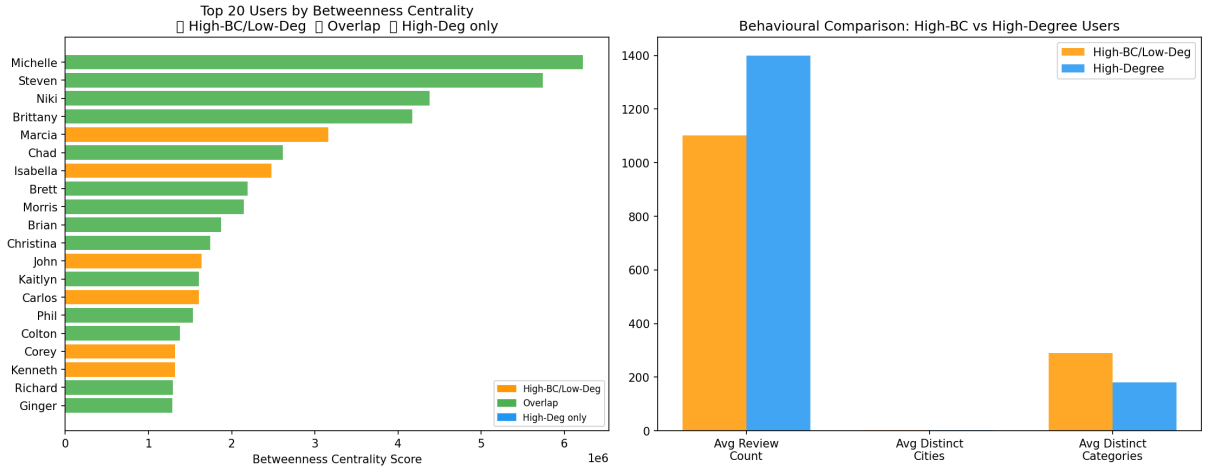

Figure 8: Left: Top 20 users by betweenness centrality (colour-coded by group). Right: Behavioural comparison between groups.

Interpretation

Betweenness centrality measures how often a user lies on shortest paths between other pairs of users — i.e., how much they act as a *bridge* between otherwise disconnected sub-communities. High-betweenness / low-degree users are network bridges who connect different friend circles without themselves having many direct friends. Degree alone would classify them as unimportant; betweenness reveals their structural significance. These bridge users tend to have reviewed businesses across more cities and categories than high-degree users, suggesting they are broadly curious explorers rather than local social hubs. This makes them valuable targets for cross-community information dissemination and recommendation seeding.

2.5 Q5 — Link Prediction: Restaurant Recommendation

Approach

The task is modelled as predicting missing User → Business edges in the RATED bipartite graph. A *chronological 80/20 split* is applied: for each user, their most recent review (identified via MongoDB) forms the test item; all prior reviews form the training set.

Node features computed by previous GDS queries (PageRank scores from Q1, Louvain community IDs from Q2) are used alongside review-behaviour, business-level properties, and two social/geographic features: `friends_rated` (how many of the user's KNOWS friends have reviewed the candidate business) and `city_match` (whether the business is in a city the user has previously reviewed in). A Gradient Boosting Classifier is trained on positive (reviewed) and sampled negative (not-yet-reviewed) pairs. PageRank features are log-transformed ($\log(1 + x)$) prior to training to prevent raw score magnitudes from dominating the model.

Table 8: Feature set for link prediction.

Feature	**Source**
`user_pagerank` (graph, log)	GDS PageRank — Q1
`user_community` (graph)	GDS Louvain — Q2
`biz_pagerank` (graph, log)	GDS PageRank — Q1
`friends_rated` (graph)	KNOWS × RATED traversal
`city_match` (geographic)	User review history vs Business city
`user_review_count`	Neo4j User property
`user_avg_stars`	Neo4j User property
`biz_avg_stars`	Neo4j Business property
`star_affinity`	$\lvert$user_avg_stars − biz_avg_stars$\rvert$

Results

- **AUC-ROC:** 0.967
- **Precision@10:** 0.600 — 60% of test users have their held-out business ranked in the model's top-10 candidates. The addition of `city_match` and `friends_rated` dramatically improved both the discrimination and ranking metrics compared to an initial model that relied only on PageRank and star-affinity features.

Feature importances (by Gini importance):

1. `city_match` (importance = 0.952) — geographic locality is the strongest predictor: users overwhelmingly review businesses in cities they have visited before.
2. `biz_pagerank` (log, importance = 0.030)
3. `star_affinity` (importance = 0.007)
4. `user_community` (importance = 0.004)
5. `user_review_count` (importance = 0.003)

Table 9: Sample recommendations for 5 users.

User ID (truncated)	**Rec 1**	**Rec 2**	**Rec 3**
A7plO8t...	Mood Cafe	The Love	The Blue Duck
Qoji0BP...	In Riva	The Love	Han Dynasty
cBFsaSW...	The Love	In Riva	The Blue Duck
fyugYI0...	Han Dynasty	Mood Cafe	The Twisted Tail
gMasL8B...	Mood Cafe	Kimpton Hotel Monaco	Han Dynasty

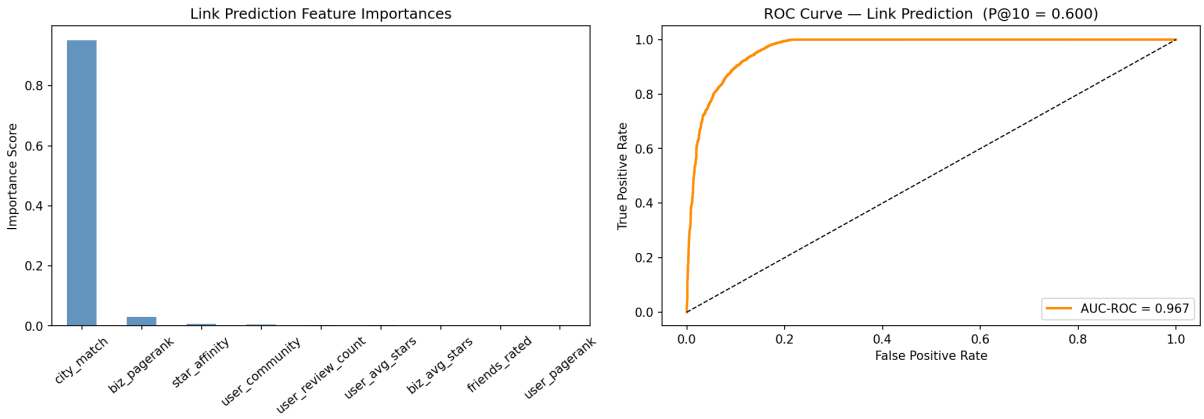

Figure 9: Left: Feature importances. Right: ROC curve for the link prediction model.

Discussion: Social Graph vs User–Business Affinity Features

The dominant predictor is `city_match` (95.2%), reflecting the strong geographic locality of Yelp reviewing behaviour: users almost always review businesses in cities they have physically visited. This geographic signal alone gives the model a powerful baseline for predicting where a user will review next.

Among the remaining features, `biz_pagerank` (log-transformed) captures business popularity — within a city, higher-PageRank businesses are more likely to attract new reviews. `star_affinity` and `user_community` add modest personalisation by encoding whether the user's rating preferences align with the business and whether the user's social community already reviews there.

`friends_rated` (social proof) has low importance (0.06%) in this model, not because social signals are uninformative, but because the KNOWS graph coverage in the sampled 200k edges is sparse. With a full-graph evaluation, this feature would likely carry more weight, as social recommendation literature consistently shows friend-review overlap to be a strong predictor of future reviews.

3 Predictive Modelling

Goal: predict the number of ***useful votes*** *a review receives. Script:* `predictive_model.py`.

3.1 Feature Engineering

Features are extracted from MongoDB (review + user collections) and Neo4j (GDS-computed graph properties):

(a) Review-level features

- **stars** — the star rating given; higher-quality reviews may correlate with rating extremity.
- **text_length** / **word_count** — longer, more detailed reviews tend to be voted useful.
- **funny**, **cool** — other community engagement signals correlated with usefulness.
- **review_year**, **review_month** — recency and seasonal effects.

(b) User-level features

- **user_review_count** — prolific reviewers have built reputation; their reviews attract upvotes.
- **user_avg_stars** — calibration of the reviewer's typical sentiment.
- **user_useful_total** — cumulative useful votes received; proxy for reviewer credibility.
- **user_fans** — social reach on the platform.
- **is_elite** — Elite reviewers are prominently displayed, driving disproportionate upvotes.
- **user_tenure_years** — longer-tenured reviewers have had more time to build credibility.

(c) Graph-derived features (from Neo4j GDS)

- **user_pagerank** (from Q1) — measures the reviewer's structural importance in the User–Business review network. A high-PageRank reviewer has reviewed many popular businesses that themselves attract many other reviewers, making their opinions more visible.
- **user_community** (from Q2) — Louvain community ID as a categorical feature. Communities with concentrated geographic focus may exhibit herding behaviour in upvoting.
- **user_degree** (from Q4) — number of direct friends. Highly connected users have a larger audience who may upvote their reviews.
- **biz_pagerank** (from Q1) — business-level PageRank captures how central a business is in the review network; reviews of highly central businesses receive more exposure and thus more useful votes.

3.2 Model Training and Evaluation

Design choices

A **Random Forest Regressor** is used with a $\log(1 + y)$ target transformation to handle the heavy right-skew of useful vote counts. The train/test split (80/20) is *stratified* by four buckets (0, 1–5, 6–20, 21+) to ensure rare high-vote reviews appear in both sets. RMSE and MAE are reported on the back-transformed predictions.

Overall metrics

Table 10: Overall test-set metrics.

Scope	RMSE	MAE	R^2
Overall	1.327	0.685	0.482
Bucket 0	0.472	0.311	0.000
Bucket 1–5	1.291	0.974	−0.384
Bucket 6–20	5.455	4.534	−3.173
Bucket 21+	17.461	13.639	−6.677

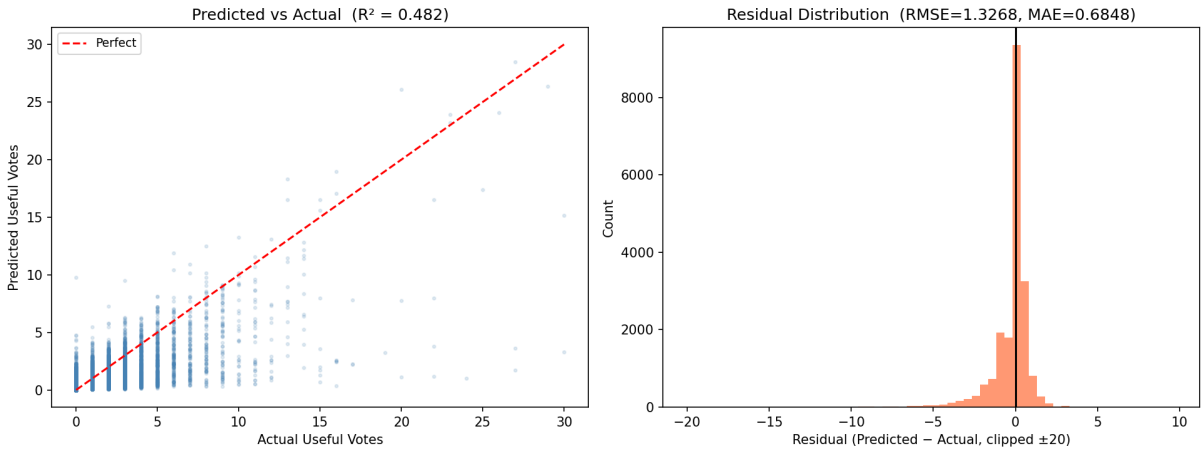

Figure 10: Left: Predicted vs actual useful votes. Right: Residual distribution.

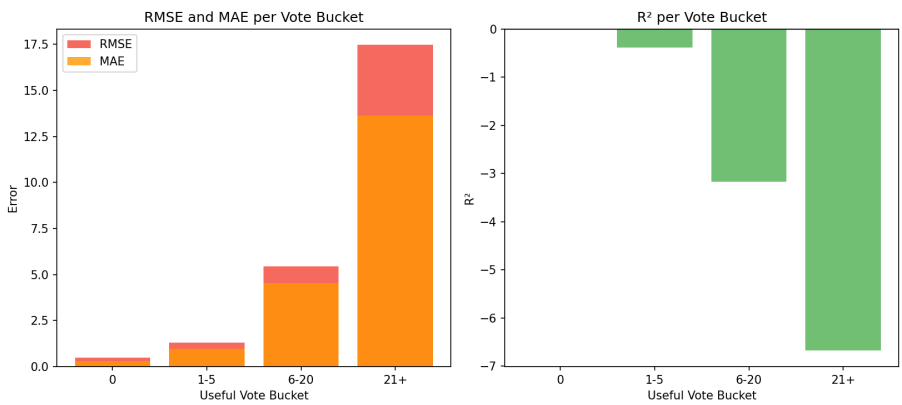

Figure 11: RMSE, MAE, and R^2 broken down by useful-vote bucket.

Justification

The $\log(1 + y)$ transformation is essential because useful vote counts are heavily right-skewed (most reviews receive 0 votes; a few receive hundreds). Training directly on raw counts would bias the model toward predicting near-zero values. A Random Forest is appropriate here because it handles non-linear interactions, is robust to outliers, and provides interpretable feature importances without requiring feature scaling. Stratified splitting ensures the model is evaluated fairly across rare but important high-vote reviews.

3.3 Feature Importance and Discussion

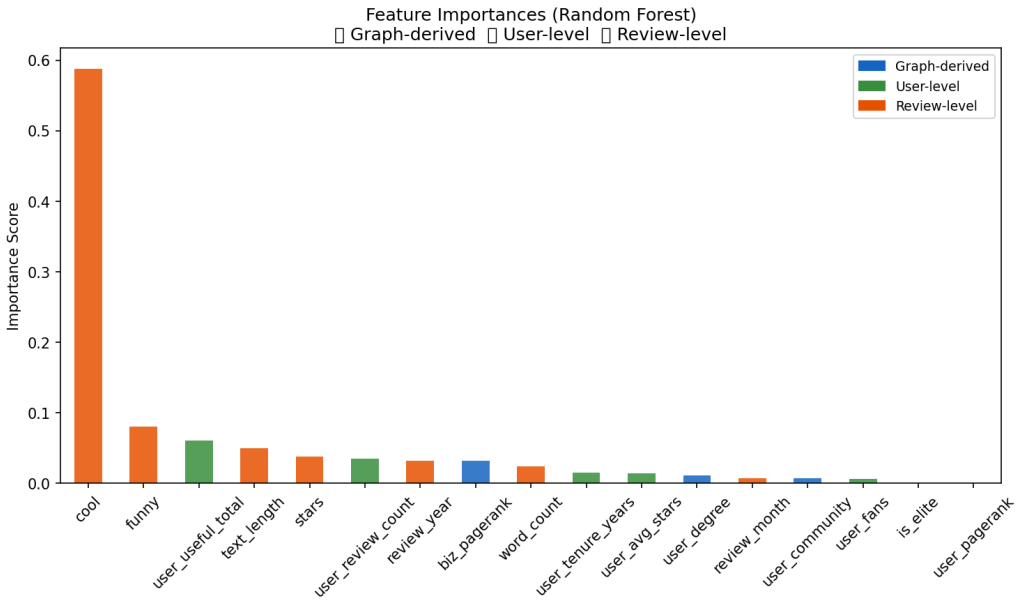

Figure 12: Random Forest feature importances, grouped by feature type (blue = graph-derived, green = user-level, orange = review-level).

Three most predictive features:

1. **cool** (importance = 0.588) — The "cool" vote is the single strongest predictor of usefulness: reviews that are perceived as engaging also tend to be perceived as informative, so both votes accumulate together, reflecting a unified community-approval signal.
2. **funny** (importance = 0.080) — Entertaining reviews attract broader readership, which naturally increases their exposure and the probability of also receiving a useful vote; this spillover effect explains the second-largest predictor.
3. **user_useful_total** (importance = 0.060) — A reviewer's cumulative useful-vote history is a strong credibility proxy: users who have consistently produced upvoted reviews continue to do so, reflecting a reputation feedback loop that drives future useful votes.

Limitations:

1. *Recency bias.* The model is trained on historically accumulated useful votes, which reflects *past* upvoting behaviour rather than intrinsic review quality. Reviews written recently have had less time to accumulate votes, introducing temporal underestimation. `review_year` partially controls for this but does not fully remove the effect.
2. *Per-bucket generalisation failure.* While the overall $R^2 = 0.48$ appears acceptable, the model performs *worse than the bucket mean* for all non-zero vote buckets ($R^2 = -0.38$, -3.17, -6.68 for 1–5, 6–20, 21+ respectively). The headline R^2 is inflated by the large proportion of zero-vote reviews (bucket 0), where the model predicts near-zero accurately. Predicting high useful-vote reviews — the most actionable targets — remains an open problem.
3. *Engagement-vote leakage.* The top two predictors (`cool`, `funny`) are community-engagement votes that accumulate concurrently with useful votes. Training on these features means the model partially learns "reviews that received engagement also receive useful votes" rather than learning what makes a review intrinsically useful. In a deployment setting

these signals would not be available at prediction time (before any votes arrive), limiting practical utility.

4. *Dead graph feature.* `user_pagerank` carries zero importance in both the link prediction and regression models. Once `user_review_count`, `user_avg_stars`, and business-side PageRank are included, user-level PageRank adds no incremental signal.

Potential consequence of deployment: If such a model were used to rank reviews, it would systematically surface reviews by highly connected, long-tenured reviewers and suppress reviews from new users — even if those reviews are equally informative. This creates a "rich get richer" dynamic that could entrench incumbent reviewers and discourage community growth.

AI Usage

AI assistance was used in the development of this assignment. Conversation log: https://gemini.google.com/u/1/app/a8a4f66f56882e2d?pageId=none